



Supply Chain Risk & Decision Intelligence Copilot

Principal Data Scientist — Architecture, Decision Intelligence & Meeting Documentation

Meeting Purpose

A consolidated, executive-ready view of the UNAI Phase 1 platform: business value, enterprise data model, multi-agent architecture, decision logic, APIs, benchmark evidence, demo flow, operating model, and documented reconciliation points.

System reference: v2.5.0 Enterprise Multi-Agent Decision Intelligence

Prepared from the four supplied UNAI documentation artifacts

1. Executive Brief

UNAI is documented as an enterprise-grade Supply Chain Risk & Decision Intelligence Copilot designed to sense disruptions, cascade their impact through multi-tier supply networks, quantify operational and commercial exposure, and present mitigation trade-offs through an orchestrated multi-agent architecture.

Executive value proposition

Convert fragmented supply-chain signals and enterprise ledgers into a traceable decision workflow: Detect → Expose → Quantify → Monetize → Simulate → Explain.

Capability	What the platform provides	Business question answered
Disruption sensing	Live news, weather and seismic inputs plus disruption signals	What is happening and where?
Network exposure	Multi-tier BOM traversal and plant dependency mapping	Which parts, plants and products are exposed?
Operations intelligence	Capacity loss, utilization/redundancy and recovery horizon	How much production capability is at risk?
Inventory intelligence	Days of Supply, in-transit inventory and safety-stock breach checks	When will material availability become critical?
Commercial impact	Revenue at risk, OTIF degradation, fill rate and SLA penalties	What is the financial/customer blast radius?
Decision intelligence	Four mitigation strategies with residual risk, protected revenue and strategy cost	What trade-offs can decision-makers evaluate?
Copilot	Fast-path and multi-agent telemetry synthesis	Can an executive ask why a result occurred and get an evidence-based answer?

2. Platform Evolution: Prototype → Enterprise PoC

The KT material describes a significant evolution from a small prototype into a production-oriented enterprise PoC architecture.

Dimension	Prototype baseline	Enterprise PoC state
Agent topology	3 macro agents	9 specialized/autonomous components organized across 5 orchestrated teams
Data scale	~50 mock records	10 enterprise flat files / 100k+ records described in KT
Data access	Sequential per-request reads	EnterpriseDataLoader with in-memory hash indexes
BOM	Single-tier lookup	Multi-tier dynamic traversal across 44,896 BOM linkages
Manufacturing	Macro plant approximation	120 MES production lines across 20 plants
Inventory	Estimated consumption	30,000-record inventory ledger with DOS/in-transit/safety-stock fields
Sourcing	Synthetic vendor generation	15,869 alternate-source mappings
Commercial	10 mock orders	10,000 customer accounts + 9,000 finished goods
Disruption arbitration	Single-event isolation	Cross-signal conflicts + cumulative and time-phased impact datasets
UI	Basic tables	Executive control tower, pipeline tracker, topology canvas, live radar, theme engine
Verification	5 simple unit tests	KT claims 23 unit/integration tests;

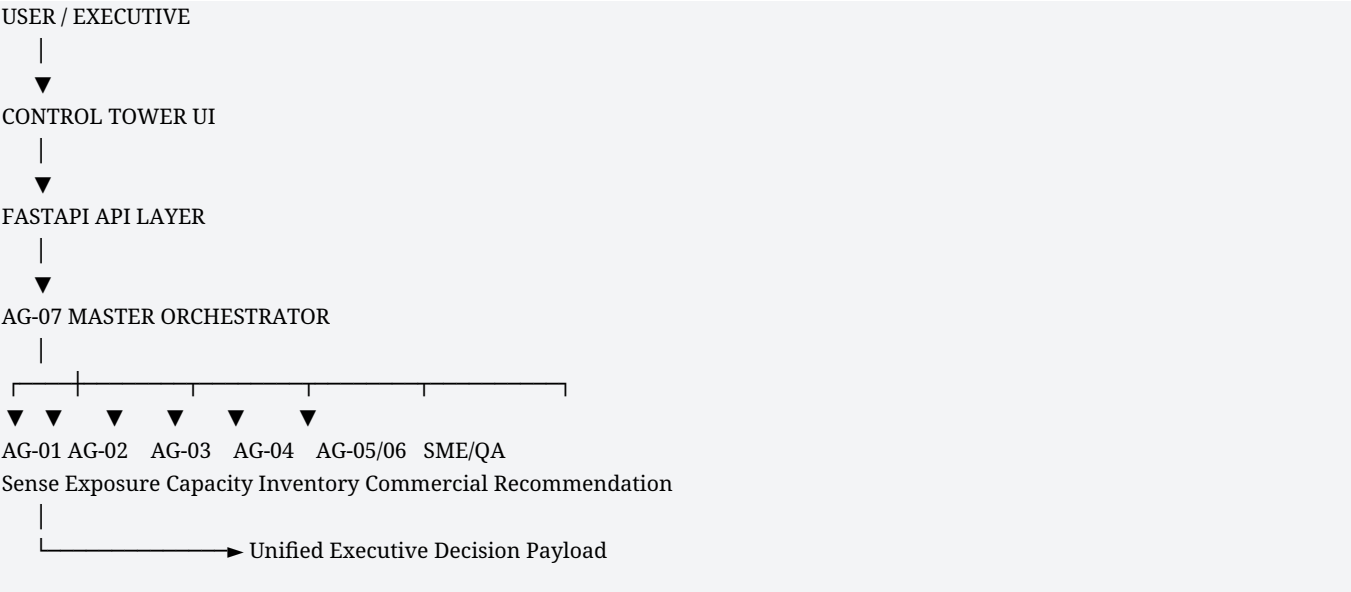
		core project doc claims 16 tests
--	--	----------------------------------

Meeting interpretation

The architecture is not just an LLM wrapper. The core value is deterministic enterprise data processing and domain calculations, with LLM reasoning positioned around strategic trade-off generation and conversational synthesis.

3. Target Architecture & Agent Operating Model

The supplied architecture documents describe a hierarchical multi-agent system. AG-07 acts as the master orchestrator, coordinating domain agents and synthesizing the executive payload. The operational flow is represented as five visible stages in the control tower.



Agent	Primary responsibility	Key inputs / data	Primary output
AG-01 Sensing	Detect and normalize disruption signals	Tavily, Marketaux, Open-Meteo, USGS, time-phased signals	Severity score, category, headlines, shortages
AG-02 Exposure	Trace supplier → part → FG → plant exposure	BOM, finished goods, supplier ledgers	Affected parts, FGs, plants, topology
AG-03 Capacity	Quantify production capacity loss	Capacity telemetry, exposed plants, severity	Capacity loss %, redundancy effect, recovery
AG-04 Inventory	Determine material runway and stockout risk	Inventory ledger, affected parts, recovery horizon	DOS, safety-stock breaches, stockout risk
AG-05 Recommendation	Generate mitigation trade-offs	Exposure, capacity, inventory, financials, alternate sources	Four mitigation strategies
AG-06 Customer/Financial	Monetize commercial exposure	Customers, FGs, orders/financial ledgers	Revenue at risk, OTIF, fill rate, SLA penalties
AG-07 Manager	Orchestrate and arbitrate	All agent telemetry + conflict/cumulative datasets	Unified executive payload
Q&A Copilot	Explain results conversationally	Pipeline telemetry + cached domain context	Fast-path or LLM-assisted answer

4. Five-Stage Decision Pipeline

1. Event Intelligence — establish what disruption signal exists and its normalized severity.
2. Exposure & BOM — identify the affected supply-network nodes, parts, finished goods and plants.
3. Operations Capacity — quantify production loss, utilization/redundancy effects and recovery horizon.
4. Commercial Impact — translate operational exposure into customer, OTIF, revenue and contractual consequences.
5. Strategy Recommendation — compare mitigation options and expose the economic/risk trade-offs.

Design principle

Each stage should produce structured telemetry that can be inspected, tested and passed forward. The orchestrator should synthesize those outputs rather than asking a language model to infer core numeric facts from raw files.

5. Enterprise Data Architecture

The KT artifact describes 10 production flat files, while the earlier end-to-end project documentation describes a 6-dataset core model. For meetings, this should be presented as an evolution in the data contract rather than silently merged.

Dataset	Scale / role	Important fields
bom.csv	44,896 records; multi-tier engineering hierarchy	fg_id, part_id, qty_per, bom_level, part_primary_vendor_id
finished_goods.csv	9,000 records; product economics and plant mapping	fg_id, program, fg_family, plant_id, price_usd, std_cost_usd, margin_pct, weekly_volume
inventory.csv	30,000 records; plant/part stock ledger	plant_id, part_id, on_hand_qty, days_cover, in_transit_qty, safety_stock_qty
capacity.csv	120 production lines across 20 plants	plant_id, line_id, fg_family, weekly_capacity_units, utilization_pct, changeover_hours, status
customers.csv	10,000 customer accounts	customer_id, tier_rank, otif_target_pct, penalty_usd_per_day, annual_revenue_usd, contract_type
alt_sources.csv	15,869 qualified alternate-source mappings	part_id, alt_vendor_id, lead_time_weeks, unit_cost_delta_pct, ppap_weeks, capacity_available_pct
cross_signal_conflicts.csv	3 conflict rules	conflict_id, signal_a, signal_b, contested_resource, resolution_options
cumulative_impact.csv	4 cumulative-impact records	fg_at_risk_deduped, plants_hit, customers_exposed, open_so_musd, otif_trough_pt
impact_timephased.csv	105 time-phased records	signal_id, impact_domain, unit, week_bucket, value
agents.csv	7 governance records	agent_id, role, primary_inputs, primary_outputs, systems_touched, autonomy

5.1 Legacy/Core Six-File Contract

Dataset	Role
suppliers.csv	Supplier registry, geography, tier and parent-supplier relationships
bom.csv	Part/product/supplier relationships and criticality

plant_capacity.csv	Plant capacity and redundancy
plant_parts.csv	Plant-to-part consumption/buffer relationships
inventory.csv	Stock, safety stock, burn rate and lead time
customer_orders.csv	Order, revenue, due-date and SLA penalty information

Reconciliation required before production handover

The documents use two overlapping schemas. Before implementation is frozen, establish one canonical enterprise data contract and explicitly map legacy fields (e.g., customer_orders) to the newer ledger model (e.g., customers + finished_goods + cumulative impact).

6. Data Access & Performance Architecture

The KT guide describes EnterpriseDataLoader as an in-memory indexing layer intended to eliminate repeated disk I/O and provide constant-time lookups for common access patterns.

EnterpriseDataLoader

- └─ bom_by_fg[fg_id]
- └─ alt_sources_by_part[part_id]
- └─ capacity_by_plant[plant_id]
- └─ customers_by_id[customer_id]
- └─ timephased_by_signal[signal_id]

The documented design target is sub-10ms API responses for indexed data access. The benchmark artifact, however, reports approximately 983 ms for initial 100k+ record indexing and approximately 1.59 seconds for an end-to-end orchestrated run. These are different measurements: startup/index construction versus full pipeline execution.

Metric	Documented benchmark	Interpretation
100k+ record ingestion/indexing	983.48 ms	Initial in-memory index construction in the benchmark run
AG-01 sensing	1350.28 ms	External/live sensing dominates the benchmark
AG-02 BOM traversal	150.91 ms	Large multi-tier traversal
AG-03/04 capacity + inventory	0.13 ms	Deterministic in-memory calculations
AG-06 commercial math	0.50 ms	Deterministic financial calculation
AG-05/07 decision	360.18 ms	Trade-off/decision stage
Master end-to-end	1594.33 ms	Full orchestrated benchmark run
Copilot synthesis	0.01 ms	Documented fast-path synthesis benchmark

7. Decision Intelligence & Mathematical Logic

7.1 Event Sensing

- Market/news sensing through Tavily and Marketaux.
- Weather signals through Open-Meteo, including wind, precipitation and temperature anomalies.
- Seismic signals through USGS; the project documentation specifies magnitude >4.0 within 300 km of a supplier facility.
- Signals are normalized into a disruption severity score from 0–100 and categorized by event type.

7.2 Exposure Cascading

6. Identify the disrupted primary supplier.
7. Traverse BOM relationships to affected components.
8. Map affected finished goods and assembly plants.
9. Construct supply-network topology nodes and links for visualization and downstream calculations.

7.3 Capacity Loss

Capacity = Σ weekly_capacity_units
 Redundancy = $(100 - \text{avg_utilization}) / 100$
 Loss = Exposure Fraction $\times$ Severity/100 $\times$ (1 - Redundancy)

7.4 Inventory Runway

DOS = Current Stock / Daily Usage Rate
 Safety-stock breach = on_hand_qty < safety_stock_qty
 Expected stockout = DOS < recovery_days

7.5 Commercial Impact

Penalty = penalty_usd_per_day $\times$ delay_days
 Revenue at Risk = $\Sigma(\text{weekly volume} \times \text{price} \times \text{downtime weeks})$
 Projected OTIF = baseline OTIF - OTIF trough point

7.6 Strategic Trade-off Model

Remaining Risk = Base Exposure $\times$ (Post-Mitigation Risk % / Baseline Risk %)
 Revenue Protected = Base Exposure - Remaining Risk
 Net Benefit = Revenue Protected - Strategy Cost

The documentation describes four strategies: baseline/unmitigated, logistics expedite/priority air freight, strategic alternate supplier allocation, and dual-source volume split/buffer stocking. The project documentation labels Option 3 as recommended, while the KT guide describes the options without asserting a universal winner. For governance, the recommendation should remain scenario-specific and evidence-backed.

8. Copilot & LLM Role

The architecture positions LLM capability primarily in the Recommendation SME Agent and the conversational Q&A Copilot. The documentation describes Groq Llama-3-70B for strategic trade-offs and a three-tier/zero-token fast path for common Copilot questions.

Layer	Responsibility	Preferred behavior
Deterministic domain layer	Data joins, traversals, calculations, validation	No LLM required; exact and testable
Fast-path Copilot	Answer known domain questions from current telemetry/cached context	Use zero-token/cached synthesis where applicable
LLM reasoning	Explain trade-offs, generate structured strategic options	Constrain with numeric telemetry and explicit schemas
Executive presentation	Translate structured outputs into readable explanations	Never invent numeric values not present in telemetry

Principal Data Scientist design recommendation

Keep the system numerically deterministic at the bottom of the stack and probabilistic only at the explanation/strategy layer. This reduces hallucination risk, improves reproducibility, and makes the architecture easier to validate with automated tests.

9. Control Tower Experience

UI element	Purpose
5-step orchestration bar	Makes agent execution visible and understandable to executives
Blob-style network topology	Visualizes Tier-2 material basins, Tier-1 hubs, assembly super-plants and demand market
Supplier news radar	Shows upstream news/raw-material signals and vendor sentiment
Operations scorecards	Surfaces capacity loss, DOS and recovery horizon
Commercial scorecards	Surfaces revenue at risk, OTIF/fill-rate impact and SLA penalties
Strategy leaderboard	Lets users inspect four mitigation trade-offs and comparison deltas
Copilot drawer	Allows natural-language Q&A over the current scenario
Theme engine	Provides dark glassmorphism and documented WCAG AAA light mode

10. Golden Demonstration Scenarios

Scenario	Inputs	Meeting narrative
A — Semiconductor crisis	TSMC; Hsinchu → Austin; Ocean Freight	Earthquake/seismic exposure → BOM mapping → plant capacity loss → 8.3-day DOS vs 24.8-day recovery → mitigation trade-off
B — Consumer electronics	Foxconn; Yantian/Shenzhen → Long Beach; Ocean Freight	Supplier news + substrate availability → strategic customer/commercial exposure
C — Multi-signal conflict	SIG-2026-0912 + SIG-2026-0913	Two disruptions compete for the same trans-Pacific air lane; show arbitration and de-duplicated cumulative impact
Healthy baseline	Test Healthy	Demonstrate that the platform can return zero revenue at risk / baseline health rather than creating false alarms

Five-minute executive storyline

Start with the control tower → trigger a disruption → show the pipeline progressing → reveal network exposure → show operational and inventory runway → quantify commercial impact → compare mitigation strategies → ask the Copilot why the result occurred → return to healthy baseline.

11. API & Integration Surface

Endpoint	Method	Purpose
/api/pipeline	GET	Executes the full multi-agent orchestration pipeline

/api/topology	GET	Returns graph nodes, links and cluster data
/api/suppliers/suggest	GET	Fuzzy supplier autocomplete / registry lookup
/api/chat	POST	Copilot Q&A synthesis
/api/disruptions	—	Disruption route referenced in architecture HLD
/api/financial	—	Financial route referenced in architecture HLD
/api/upload/{dataset}	POST	CSV dataset replacement/upload
/docs	GET	FastAPI interactive Swagger documentation

```
python -m uvicorn backend.main:app --host 127.0.0.1 --port 8000 --reload
python -m pytest tests/ -v
# or, per KT runbook:
python -m pytest tests/test_agents.py -v
```

12. QA, Verification & Governance

The project documentation states that 16 unit, integration and endpoint tests validate sensing, BOM exposure, capacity/inventory calculations, customer impact, mitigation trade-offs, Copilot reasoning and CSV upload endpoints. The KT document separately states 23 automated unit/integration tests with 100% passing. The benchmark runbook reports 8/8 SLA checks passed.

Verification area	Documented evidence
Core domain logic	Tests cover sensing, BOM traversal, capacity, inventory, commercial calculations
Decision strategies	Tests cover four mitigation trade-offs
Copilot	Fast-path and LLM reasoning are included in test scope
Upload	Six CSV upload endpoints are included in the project-doc test scope
Benchmark	8/8 SLA checks passed in the benchmark artifact
Regression defense	KT guide claims 23 automated integration/unit tests, 100% passing

12.1 Reconciliation Matrix

Topic	Document A	Document B	Action before formal release
Agent count	9 specialized agents/components / AG-01–AG-07 + Copilot/Simulator framing	8 explicitly listed in core hierarchy	Define canonical component taxonomy and naming
Dataset count	10 production flat files	6 core datasets	Publish canonical data contract + migration mapping
Tests	23 automated tests, 100% passing	16 tests	Publish one authoritative test inventory
Recommendation	Four options; KT avoids universal ranking	Option 3 marked recommended	Make recommendation scenario/data dependent and explain criteria
Performance	Sub-10ms lookup target	~1.59s end-to-end benchmark	Separate lookup SLA, cold start, live I/O and E2E SLA

Important

These are documentation inconsistencies, not necessarily implementation defects. They should be resolved by checking the current repository/code and producing a single version-controlled architecture and test baseline.

13. Productionization Roadmap

Workstream	Current documented state	Next engineering gate
Data contract	Multiple documented schemas	Canonical schema, validation, lineage and versioning
Ingestion	Flat files + in-memory indexing	Incremental refresh, data quality checks, observability
Agent orchestration	Hierarchical orchestration described	Typed agent contracts, retries, timeouts, idempotency
LLM layer	Groq Llama-3 decision engine + Copilot	Model gateway, prompt/version registry, structured outputs, evaluation harness
Decision governance	Strategy comparison outputs	Explainability, assumptions, confidence/uncertainty, human approval gates
External signals	Tavily/Marketaux/Open-Meteo/USGS	Rate-limit handling, source reliability, caching and fallback
Security	PoC/local endpoints documented	Authentication, authorization, secrets management, audit logging
Scalability	In-memory local PoC	Distributed cache/store and scalable data-serving layer
Observability	Benchmark outputs	Trace IDs, per-agent latency, data freshness, model/token/cost telemetry
Testing	Unit/integration tests documented	Golden datasets, scenario regression, adversarial tests, load tests

14. Principal-Level Meeting Talking Points

- The architecture separates sensing, deterministic domain analytics, orchestration and LLM-assisted decision explanation.
- The key differentiator is the ability to cascade a disruption from supplier signals through BOM, plant operations, inventory and customer economics.
- The in-memory index layer is designed to keep enterprise ledger lookups fast enough for interactive decision workflows.
- The decision engine is not expected to calculate raw truth from scratch; it consumes structured telemetry and generates explicit mitigation trade-offs.
- The Copilot is designed to explain the current scenario, including why a strategy has a particular residual risk, cost and revenue protection profile.
- Multi-signal arbitration is important because independent disruptions can compete for the same constrained resource and double-count the same finished goods.
- The benchmark demonstrates that deterministic stages can be extremely fast, while live sensing and end-to-end orchestration carry most of the latency.
- Before production sign-off, the team should reconcile the documented agent count, dataset count, test count and canonical schemas.

15. Executive Q&A Preparation

Likely question	Answer grounded in the supplied documentation
What does UNAI actually solve?	It turns disruption signals into quantified supply-network, operational, inventory and commercial impact, then compares mitigation strategies.
Why multiple agents?	Each domain has a distinct data contract and calculation responsibility; the master agent coordinates them into one decision flow.
Where is the LLM used?	Primarily in strategic trade-off generation and conversational synthesis; core numeric calculations are documented as deterministic logic.
How does it avoid false alarms?	The demo includes a healthy baseline scenario and the platform combines live/parametric signals with enterprise ledgers before generating downstream impact.
How does it handle overlapping disruptions?	Cross-signal conflict rules, cumulative impact and time-phased impact datasets support arbitration and de-duplication.
What makes the demo credible?	The runbook provides golden scenarios, benchmark timings, API endpoints and automated verification instructions.
What must happen before production?	Canonicalize data/agent/test contracts, add security and observability, and validate the current implementation against the documentation.

Appendix A — Source Traceability

This meeting document consolidates the supplied source artifacts without introducing external research. Terminology and claims are retained where possible; where the artifacts differ, the difference is explicitly called out rather than silently reconciled.

Source artifact	Primary contribution
KNOWLEDGE_TRANSFER_AND_ARCHITECTURE_EVOLUTION.md	Architecture evolution, 10-file enterprise dataset inventory, agent contracts, indexing approach, UI innovations and operational runbook.
PROJECT_DOCUMENTATION.md	End-to-end architecture, business use cases, core agent hierarchy, formulas, legacy six-file schema, APIs and testing scope.
POC_BENCHMARK_AND_DEMO_RUNBOOK.md	Five-minute demo script, golden scenarios, benchmark timings, SLA result and API quick reference.
USER_GUIDE.md	Operator workflow, dashboard usage, telemetry interpretation, strategy cards, Copilot, CSV upload and theme switching.

Source boundary

No external sources were used in preparing this document. Any item marked as a reconciliation point is derived from differences between the supplied artifacts and should be validated against the current repository implementation.